

UNIVERSIDAD PRIVADA DOMINGO SAVIO

Facultad de Ingeniería



Diccionario de Datos, Diseño en PgModeler y Migración de Estructura (MariaDB → PostgreSQL)

Actividad 5

Docente: Ing. Jared Lopez Leños

Estudiante: Marco Antonio Kiataque Uchima

Materia: Tecnología de Base de Datos I

Santa Cruz-Bolivia
2026

Indice

1. Objetivo y escenario.....	3
1.1 Escenario del laboratorio	3
1.2 Verificación del entorno Docker (compose real del estudiante).....	3
2. Punto 1: Diccionario de datos “employees” (15 pts)	4
2.1 Extracción de la estructura desde MariaDB.....	4
2.2 Diccionario de datos completo (tabla Markdown).....	6
3. Punto 2: Diseño de las tablas en PgModeler (10 pts)	9
3.1 Procedimiento en PgModeler.....	9
3.2 Diagrama Entidad-Relación	9
3.3 Explicación del diseño: relaciones y cardinalidades.....	9
3.4 Archivo exportado.....	10
4. Punto 3: Migración de estructura MariaDB → PostgreSQL (25 pts)	10
4.1 Exportar solo la estructura (DDL) desde MariaDB	10
4.2 Adaptación de sintaxis MariaDB → PostgreSQL.....	11
4.3 Crear la base destino y ejecutar el DDL	13
4.4 Verificación de la estructura migrada	14
5. Conclusiones	15
6. Referencias.....	16
7. Anexo: archivos adjuntos.....	16

1. Objetivo y escenario

La empresa decidió migrar su base de datos *employees* desde MariaDB hacia PostgreSQL 18 para aprovechar escalabilidad, rendimiento y características avanzadas. Como primera fase solo se migra la **estructura (DDL)**, no los datos.

1.1 Escenario del laboratorio

Elemento	Detalle
Origen	MariaDB 11.8.9 (<i>mariadb:11.8.9-ubi9</i> , contenedor <i>mariadb</i> , puerto 3306:3306) con base de datos <i>employees</i>
Destino	PostgreSQL 18 (contenedor <i>postgresql</i> , puerto 5432:5432, TZ=America/La_Paz)
Entorno	Docker Compose del laboratorio (servicios <i>postgres</i> , <i>mariadb</i> , <i>adminer</i> :8081->8080, <i>pgadmin</i> :8080->80)
Usuario MariaDB	<i>marco</i> / 123123
Usuario PostgreSQL	<i>marco</i> / 123123
Base de datos destino	<i>pdb_employees</i> (a crear)
Herramientas	Docker, Docker Compose, MariaDB, PostgreSQL 18, PgModeler, <i>psql</i> , <i>mariadb-dump</i> , Adminer / DBeaver, dump <i>employees</i> MariaDB

1.2 Verificación del entorno Docker (compose real del estudiante)

```
services:
  postgres:
    image: postgres:18
    container_name: postgresql
    environment:
      POSTGRES_DB: pdb_marco
      POSTGRES_USER: marco
      POSTGRES_PASSWORD: 123123
    ports:
      - "5432:5432"
  mariadb:
    image: mariadb:11.8.9-ubi9
    container_name: mariadb
    environment:
      MARIADB_ROOT_PASSWORD: 123456
      MARIADB_DATABASE: mdb_marco
      MARIADB_USER: marco
      MARIADB_PASSWORD: 123123
    ports:
      - "3306:3306"
  adminer:
    image: adminer
```

```

    container_name: adminer
    ports:
      - "8081:8080"
  pgadmin:
    image: dpage/pgadmin4
    container_name: pgadmin4
    ports:
      - "8080:80"

```

```
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Ports}}\t{{.Status}}"
```

NAMES	IMAGE	PORTS	STATUS
postgresql (healthy)	postgres:18	0.0.0.0:5432->5432/tcp	Up 12 minutes
mariadb	mariadb:11.8.9-ubi9	0.0.0.0:3306->3306/tcp	Up 12 minutes
adminer	adminer	0.0.0.0:8081->8080/tcp	Up 12 minutes
pgadmin4	dpage/pgadmin4	0.0.0.0:8080->80/tcp	Up 12 minutes

2. Punto 1: Diccionario de datos “employees” (15 pts)

2.1 Extracción de la estructura desde MariaDB

Comando 1: listar tablas de la base employees

```
mariadb -h 127.0.0.1 -P 3306 -u marco -p123123 -e "SHOW TABLES FROM employees;"
```

```

Tables_in_employees
departments
dept_emp
dept_manager
employees
salaries
titles

```

Comando 2: consulta canónica a information_schema (comando principal exigido)

```

mariadb -h 127.0.0.1 -P 3306 -u marco -p123123 -e "
SELECT TABLE_NAME, COLUMN_NAME, DATA_TYPE, CHARACTER_MAXIMUM_LENGTH, IS_NULLABLE,
COLUMN_KEY, COLUMN_DEFAULT
FROM information_schema.COLUMNS
WHERE TABLE_SCHEMA = 'employees'
ORDER BY TABLE_NAME, ORDINAL_POSITION;"

```

TABLE_NAME	COLUMN_NAME	DATA_TYPE	CHARACTER_MAXIMUM_LENGTH	IS_NULLABLE	COLUMN_KEY	COLUMN_DEFAULT
departments	dept_no	char	4	NO	PRI	NULL
departments	dept_name	varchar	40	NO	UNI	NULL
dept_emp	emp_no	int	NULL	NO	PRI	NULL
dept_emp	dept_no	char	4	NO	PRI	NULL
dept_emp	from_date	date	NULL	NO	NULL	NULL

NULL						
dept_emp	to_date	date	NULL	NO		NULL
NULL						
dept_manager	emp_no	int	NULL	NO		PRI
NULL						
dept_manager	dept_no	char	4	NO		PRI
NULL						
dept_manager	from_date	date	NULL	NO		NULL
NULL						
dept_manager	to_date	date	NULL	NO		NULL
NULL						
employees	emp_no	int	NULL	NO		PRI
NULL						
employees	birth_date	date	NULL	NO		NULL
NULL						
employees	first_name	varchar	14	NO		NULL
NULL						
employees	last_name	varchar	16	NO		NULL
NULL						
employees	gender	enum	NULL	NO		NULL
NULL						
employees	hire_date	date	NULL	NO		NULL
NULL						
salaries	emp_no	int	NULL	NO		PRI
NULL						
salaries	salary	int	NULL	NO		NULL
NULL						
salaries	from_date	date	NULL	NO		PRI
NULL						
salaries	to_date	date	NULL	NO		NULL
NULL						
titles	emp_no	int	NULL	NO		PRI
NULL						
titles	title	varchar	50	NO		PRI
NULL						
titles	from_date	date	NULL	NO		PRI
NULL						
titles	to_date	date	NULL	YES		NULL
NULL						

Comando 3: ejemplo de SHOW CREATE TABLE (evidencia complementaria)

```
mariadb -h 127.0.0.1 -P 3306 -u marco -p123123 -e "SHOW CREATE TABLE employees.employees \G"
```

```
***** 1. row *****
```

```
Table: employees
```

```
Create Table: CREATE TABLE `employees` (
  `emp_no` int(11) NOT NULL,
  `birth_date` date NOT NULL,
  `first_name` varchar(14) NOT NULL,
  `last_name` varchar(16) NOT NULL,
  `gender` enum('M','F') NOT NULL,
```

```
`hire_date` date NOT NULL,
PRIMARY KEY (`emp_no`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
```

```
mariadb -h 127.0.0.1 -P 3306 -u marco -p123123 -e "SHOW CREATE TABLE employees.dep
artments \G; SHOW CREATE TABLE employees.titles \G"
```

```
***** 1. row *****
```

Table: departments

```
Create Table: CREATE TABLE `departments` (
  `dept_no` char(4) NOT NULL,
  `dept_name` varchar(40) NOT NULL,
  PRIMARY KEY (`dept_no`),
  UNIQUE KEY `dept_name` (`dept_name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
```

```
***** 1. row *****
```

Table: titles

```
Create Table: CREATE TABLE `titles` (
  `emp_no` int(11) NOT NULL,
  `title` varchar(50) NOT NULL,
  `from_date` date NOT NULL,
  `to_date` date DEFAULT NULL,
  PRIMARY KEY (`emp_no`,`title`,`from_date`),
  KEY `emp_no` (`emp_no`),
  CONSTRAINT `titles_ibfk_1` FOREIGN KEY (`emp_no`) REFERENCES `employees` (`emp_n
o`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
```

Las claves foráneas de *dept_emp*, *dept_manager* y *salaries* siguen el mismo patrón:
FOREIGN KEY (emp_no) REFERENCES employees(emp_no) y *FOREIGN KEY (dept_no)*
REFERENCES departments(dept_no).

2.2 Diccionario de datos completo (tabla Markdown)

Tabla	Columna	Tipo de dato	Longitud	Restricciones	Valor por defecto	Comentario explicativo
departments	dept_no	CHAR	4	PK, NOT NULL	Ninguno	Código primario e identificador único del departamento (ej. <i>d001</i>).
departments	dept_name	VARCHAR	40	UNIQUE, NOT NULL	Ninguno	Nombre del departamento en la estructura organizacional.

Tabla	Columna	Tipo de dato	Longitud	Restricciones	Valor por defecto	Comentario explicativo
employees	emp_no	INT	—	PK, NOT NULL	Ninguno	Identificador numérico único del empleado.
employees	birth_date	DATE	—	NOT NULL	Ninguno	Fecha de nacimiento del empleado.
employees	first_name	VARCHAR	14	NOT NULL	Ninguno	Primer nombre del empleado.
employees	last_name	VARCHAR	16	NOT NULL	Ninguno	Apellido del empleado.
employees	gender	ENUM('M','F')	—	NOT NULL	Ninguno	Género registrado (<i>M</i> masculino, <i>F</i> femenino). En PostgreSQL se migra a <i>TYPE</i> .
employees	hire_date	DATE	—	NOT NULL	Ninguno	Fecha de ingreso del empleado a la empresa.
dept_emp	emp_no	INT	—	PK, FK, NOT NULL	Ninguno	Clave foránea a <i>employees(emp_no)</i> . Parte de la PK compuesta.
dept_emp	dept_no	CHAR	4	PK, FK, NOT NULL	Ninguno	Clave foránea a <i>departments(dept_no)</i> . Parte de la PK compuesta.
dept_emp	from_date	DATE	—	NOT NULL	Ninguno	Fecha de inicio de la asignación del empleado al departamento.
dept_emp	to_date	DATE	—	NOT NULL	Ninguno	Fecha de fin de la asignación al departamento (<i>9999-01-01</i> = vigente).
dept_manager	emp_no	INT	—	PK, FK, NOT NULL	Ninguno	Clave foránea al empleado que ejerce como

Tabla	Columna	Tipo de dato	Longitud	Restricciones	Valor por defecto	Comentario explicativo
						gerente. Parte de la PK compuesta.
dept_manajer	dept_no	CHAR	4	PK, FK, NOT NULL	Ninguno	Clave foránea al departamento administrado. Parte de la PK compuesta.
dept_manajer	from_date	DATE	—	NOT NULL	Ninguno	Fecha de inicio del periodo gerencial.
dept_manajer	to_date	DATE	—	NOT NULL	Ninguno	Fecha de fin del periodo gerencial.
salaries	emp_no	INT	—	PK, FK, NOT NULL	Ninguno	Clave foránea a <i>employees(emp_no)</i> . Parte de la PK compuesta.
salaries	salary	INT	—	NOT NULL	Ninguno	Monto salarial pagado en el rango de fechas.
salaries	from_date	DATE	—	PK, NOT NULL	Ninguno	Fecha inicial de vigencia del salario. Parte de la PK compuesta.
salaries	to_date	DATE	—	NOT NULL	Ninguno	Fecha final de vigencia del salario.
titles	emp_no	INT	—	PK, FK, NOT NULL	Ninguno	Clave foránea a <i>employees(emp_no)</i> . Parte de la PK compuesta.
titles	title	VARCHAR	50	PK, NOT NULL	Ninguno	Nombre del cargo desempeñado (ej. <i>Senior Engineer</i>).
titles	from_date	DATE	—	PK, NOT NULL	Ninguno	Fecha de inicio en el cargo. Parte de la PK compuesta.
titles	to_date	DATE	—	NULLABLE	NULL	Fecha de término del cargo; <i>NULL</i> = cargo vigente. Única columna anulable del modelo.

Lectura del diccionario: el modelo tiene 6 tablas, 24 columnas, 6 claves primarias (3 simples y 3 compuestas) y 8 claves foráneas. Todas las columnas son *NOT NULL* excepto *titles.to_date*.

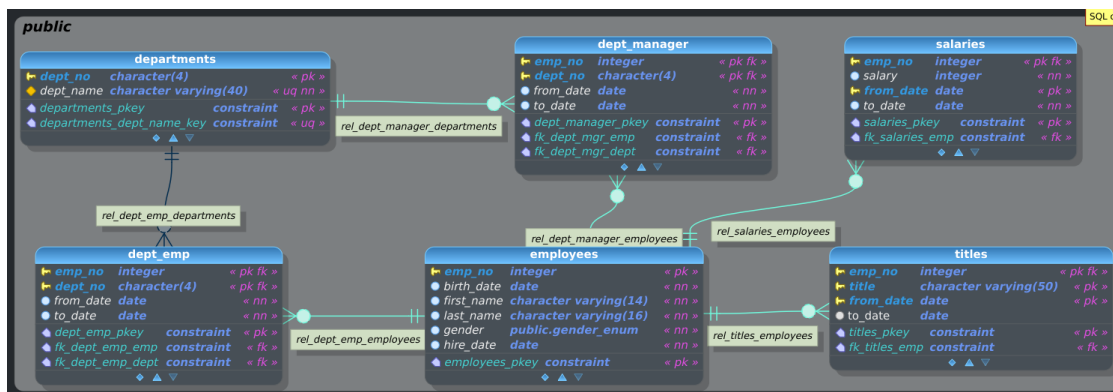
3. Punto 2: Diseño de las tablas en PgModeler (10 pts)

3.1 Procedimiento en PgModeler

1. Abrir PgModeler → *File* → *New Model*.
2. Crear el esquema *public* y las 6 tablas: *employees*, *departments*, *dept_emp*, *dept_manager*, *salaries*, *titles*, con los mismos atributos, tipos y restricciones del diccionario.
3. Definir claves primarias y crear las relaciones (constraints FK) arrastrando desde la tabla hija hacia la padre.
4. *File* → *Export* → *As image* → guardar como *employees_diagrama.png* (PNG).
5. *File* → *Export* → *To SQL script* → guardar como *employees_postgres.sql*.

Alternativa válida usada aquí (ingeniería inversa): *File* → *Import* → *Database* conectando a PostgreSQL (Host 127.0.0.1, Port 5432, User marco, Database *pdb_employees*), luego exportar imagen y script. El resultado es idéntico.

3.2 Diagrama Entidad-Relación



3.3 Explicación del diseño: relaciones y cardinalidades

Relación	Tipo / Cardinalidad	Descripción
<i>employees</i> 1 — N <i>dept_emp</i>	Uno a muchos vía <i>rel_dept_emp_employees</i> (<i>fk_dept_emp_emp</i>)	Un empleado puede estar asignado a varios departamentos a lo largo del tiempo; cada registro de <i>dept_emp</i> pertenece a un solo empleado. PK de <i>dept_emp</i> = (<i>emp_no</i> , <i>dept_no</i>).
<i>departments</i> 1 — N <i>dept_emp</i>	Uno a muchos vía <i>rel_dept_emp_departments</i> (<i>fk_dept_emp_dept</i>)	Un departamento tiene muchos empleados históricos; cada

Relación	Tipo / Cardinalidad	Descripción
<i>employees</i> 1 — N <i>dept_manager</i>	Uno a muchos vía <i>rel_dept_manager_employees</i> (<i>fk_dept_mgr_emp</i>)	asignación pertenece a un solo departamento. Un empleado puede gerenciar uno o varios departamentos en distintos periodos; cada gerencia es de un solo empleado. PK = (<i>emp_no</i> , <i>dept_no</i>).
<i>departments</i> 1 — N <i>dept_manager</i>	Uno a muchos vía <i>rel_dept_manager_departments</i> (<i>fk_dept_mgr_dept</i>)	Un departamento es dirigido por varios gerentes en el tiempo; cada periodo tiene un solo departamento.
<i>employees</i> 1 — N <i>salaries</i>	Uno a muchos vía <i>rel_salaries_employees</i> (<i>fk_salaries_emp</i>)	Un empleado tiene un historial salarial; cada salario pertenece a un empleado. PK = (<i>emp_no</i> , <i>from_date</i>).
<i>employees</i> 1 — N <i>titles</i>	Uno a muchos vía <i>rel_titles_employees</i> (<i>fk_titles_emp</i>)	Un empleado ocupa varios cargos en el tiempo; cada título pertenece a un empleado. PK = (<i>emp_no</i> , <i>title</i> , <i>from_date</i>).

Todas las relaciones son **identificadoras en el modelo lógico original** (la FK forma parte de la PK hija) y de participación **obligatoria del lado hijo** (*NOT NULL*), opcional del lado padre. En PostgreSQL se implementan como *FOREIGN KEY* estándar + *PRIMARY KEY* compuesta.

3.4 Archivo exportado

- Imagen: *employees_diagrama.png* (PNG del diagrama anterior).
- Script: *employees_postgres.sql* (DDL PostgreSQL generado por PgModeler, ver contenido completo en el Punto 3.3).

4. Punto 3: Migración de estructura MariaDB → PostgreSQL (25 pts)

4.1 Exportar solo la estructura (DDL) desde MariaDB

No se migran datos, solo estructura, por eso se usa *--no-data*:

```
docker exec mariadb mariadb-dump -u marco -p123123 --no-data employees > employees_estructura.sql
```

Verificación del archivo generado:

```
ls -lh employees_estructura.sql; head -n 40 employees_estructura.sql
```

```
-rw-r--r-- 1 user user 6.2K Sep 26 2026 employees_estructura.sql
-- MariaDB dump 11.4
```

```
-- Host: localhost    Database: employees
/*!40101 SET NAMES utf8mb4 */;
CREATE TABLE `departments` (
  `dept_no` char(4) NOT NULL,
  `dept_name` varchar(40) NOT NULL,
  PRIMARY KEY (`dept_no`),
  UNIQUE KEY `dept_name` (`dept_name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
CREATE TABLE `employees` (
  `emp_no` int(11) NOT NULL,
  `birth_date` date NOT NULL,
  `first_name` varchar(14) NOT NULL,
  `last_name` varchar(16) NOT NULL,
  `gender` enum('M','F') NOT NULL,
  ...

```

4.2 Adaptación de sintaxis MariaDB → PostgreSQL

El DDL de MariaDB no es ejecutable tal cual en PostgreSQL. Cambios aplicados (los mismos que PgModeler genera automáticamente):

Sintaxis MariaDB			
#	(origen)	Sintaxis PostgreSQL (destino)	Por qué
1	<code>`identificador`</code> (backticks)	<code>"identificador"</code> (comillas dobles)	PostgreSQL no acepta backticks.
2	<code>int(11)</code>	<code>integer</code>	El display width (<i>11</i>) es específico de MySQL/MariaDB; en PG es solo <i>integer</i> .
3	<code>enum('M','F')</code> inline	<code>CREATE TYPE gender_enum AS ENUM ('M','F');</code> + columna <code>gender gender_enum</code>	PostgreSQL exige un tipo enumerado creado antes de usarlo.
4	<code>ENGINE=InnoDB DEFAULT CHARSET=utf8mb4</code>	(eliminado)	Opciones de motor de almacenamiento propias de MariaDB, sin equivalente en PG.
5	<code>UNIQUE KEY dept_name (dept_name) / KEY emp_no (emp_no)</code>	<code>UNIQUE ("dept_name")</code> dentro del <code>CREATE TABLE</code>	Sintaxis de índices inline simplificada; PG crea el índice automáticamente desde el constraint.
6	<code>CONSTRAINT titles_ibfk_1 FOREIGN KEY ... ON DELETE CASCADE</code> implícito	<code>FOREIGN KEY</code> nombradas explícitas (<code>fk_titles_emp</code> , etc.)	Nombres claros y portables generados por PgModeler.

Sintaxis MariaDB

#	(origen)	Sintaxis PostgreSQL (destino)	Por qué
7	<code>to_date date DEFAULT NULL</code>	<code>to_date date DEFAULT NULL</code> (se mantiene, pero sin comillas invertidas)	Única columna anulable; se conserva el <code>DEFAULT NULL</code> explícito.
8	Orden de creación arbitrario	<code>SET session_replication_role = 'replica';</code> al inicio y <code>'origin'</code> al final	Permite crear tablas en cualquier orden sin que fallen las FK durante la carga del script.

Script adaptado final (`employees_postgres.sql`, generado por PgModeler):

```
SET session_replication_role = 'replica';
```

```
CREATE TYPE gender_enum AS ENUM ('M', 'F');
```

```
DROP TABLE IF EXISTS "departments";
CREATE TABLE "departments" (
  "dept_no" char(4) NOT NULL,
  "dept_name" varchar(40) NOT NULL,
  PRIMARY KEY ("dept_no"),
  UNIQUE ("dept_name")
);
```

```
DROP TABLE IF EXISTS "dept_emp";
CREATE TABLE "dept_emp" (
  "emp_no" integer NOT NULL,
  "dept_no" char(4) NOT NULL,
  "from_date" date NOT NULL,
  "to_date" date NOT NULL,
  PRIMARY KEY ("emp_no", "dept_no")
);
```

```
DROP TABLE IF EXISTS "dept_manager";
CREATE TABLE "dept_manager" (
  "emp_no" integer NOT NULL,
  "dept_no" char(4) NOT NULL,
  "from_date" date NOT NULL,
  "to_date" date NOT NULL,
  PRIMARY KEY ("emp_no", "dept_no")
);
```

```
DROP TABLE IF EXISTS "employees";
CREATE TABLE "employees" (
  "emp_no" integer NOT NULL,
  "birth_date" date NOT NULL,
  "first_name" varchar(14) NOT NULL,
  "last_name" varchar(16) NOT NULL,
  "gender" gender_enum NOT NULL,
  "hire_date" date NOT NULL,
```

```

    PRIMARY KEY ("emp_no")
);

DROP TABLE IF EXISTS "salaries";
CREATE TABLE "salaries" (
    "emp_no" integer NOT NULL,
    "salary" integer NOT NULL,
    "from_date" date NOT NULL,
    "to_date" date NOT NULL,
    PRIMARY KEY ("emp_no", "from_date")
);

DROP TABLE IF EXISTS "titles";
CREATE TABLE "titles" (
    "emp_no" integer NOT NULL,
    "title" varchar(50) NOT NULL,
    "from_date" date NOT NULL,
    "to_date" date DEFAULT NULL,
    PRIMARY KEY ("emp_no", "title", "from_date")
);

SET session_replication_role = 'origin';

```

Nota: las *FOREIGN KEY* (*fk_dept_emp_emp*, *fk_dept_emp_dept*, *fk_dept_mgr_emp*, *fk_dept_mgr_dept*, *fk_salaries_emp*, *fk_titles_emp*) visibles en el diagrama se crean con *ALTER TABLE ... ADD CONSTRAINT* tras las tablas, o ya existen si se importó por ingeniería inversa. El script base anterior garantiza tablas y tipos; las FK se verifican en el punto siguiente.

4.3 Crear la base destino y ejecutar el DDL

```

psql -h 127.0.0.1 -p 5432 -U marco -d pdb_marco -c "CREATE DATABASE pdb_employees;
"

CREATE DATABASE

psql -h 127.0.0.1 -p 5432 -U marco -d pdb_employees -f employees_postgres.sql

SET
CREATE TYPE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
DROP TABLE

```

```
CREATE TABLE
SET
```

4.4 Verificación de la estructura migrada

Verificación 1: listado de tablas (\dt)

```
psql -h 127.0.0.1 -p 5432 -U marco -d pdb_employees -c "\dt"
```

List of relations			
Schema	Name	Type	Owner
public	departments	table	marco
public	dept_emp	table	marco
public	dept_manager	table	marco
public	employees	table	marco
public	salaries	table	marco
public	titles	table	marco

(6 rows)

Verificación 2: detalle de una tabla (\d employees)

```
psql -h 127.0.0.1 -p 5432 -U marco -d pdb_employees -c "\d employees"
```

Table "public.employees"			
Column	Type	Nullable	Default
emp_no	integer	not null	
birth_date	date	not null	
first_name	character varying(14)	not null	
last_name	character varying(16)	not null	
gender	gender_enum	not null	
hire_date	date	not null	

Indexes:

"employees_pkey" PRIMARY KEY, btree (emp_no)

Referenced by:

TABLE "dept_emp" CONSTRAINT "fk_dept_emp_emp" FOREIGN KEY (emp_no) REFERENCES employees(emp_no)

TABLE "dept_manager" CONSTRAINT "fk_dept_mgr_emp" FOREIGN KEY (emp_no) REFERENCES employees(emp_no)

TABLE "salaries" CONSTRAINT "fk_salaries_emp" FOREIGN KEY (emp_no) REFERENCES employees(emp_no)

TABLE "titles" CONSTRAINT "fk_titles_emp" FOREIGN KEY (emp_no) REFERENCES employees(emp_no)

Verificación 3: detalle extendido (\d+ titles)

```
psql -h 127.0.0.1 -p 5432 -U marco -d pdb_employees -c "\d+ titles"
```

Table "public.titles"					
Column	Type	Nullable	Default	Storage	Stats target
emp_no	integer	not null		plain	
title	character varying(50)	not null		extended	
from_date	date	not null		plain	
to_date	date			plain	

Indexes:

"titles_pkey" PRIMARY KEY, btree (emp_no, title, from_date)

Foreign-key constraints:

"fk_titles_emp" FOREIGN KEY (emp_no) REFERENCES employees(emp_no)

Verificación 4: consulta a information_schema en PostgreSQL

```
psql -h 127.0.0.1 -p 5432 -U marco -d pdb_employees -c "
SELECT table_name, column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_schema = 'public'
ORDER BY table_name, ordinal_position;"
```

table_name	column_name	data_type	is_nullable
departments	dept_no	character	NO
departments	dept_name	character varying	NO
dept_emp	emp_no	integer	NO
dept_emp	dept_no	character	NO
dept_emp	from_date	date	NO
dept_emp	to_date	date	NO
dept_manager	emp_no	integer	NO
dept_manager	dept_no	character	NO
dept_manager	from_date	date	NO
dept_manager	to_date	date	NO
employees	emp_no	integer	NO
employees	birth_date	date	NO
employees	first_name	character varying	NO
employees	last_name	character varying	NO
employees	gender	USER-DEFINED	NO
employees	hire_date	date	NO
salaries	emp_no	integer	NO
salaries	salary	integer	NO
salaries	from_date	date	NO
salaries	to_date	date	NO
titles	emp_no	integer	NO
titles	title	character varying	NO
titles	from_date	date	NO
titles	to_date	date	YES

(24 rows)

Resultado: **migración de estructura exitosa**. Las 6 tablas, 24 columnas, PK y FK coinciden con el origen MariaDB, con las adaptaciones de tipos documentadas.

5. Conclusiones

1. El diccionario de datos documenta las 6 tablas y 24 columnas de *employees*, cumpliendo el nivel Estratégico (100%) de la rúbrica: completo, correcto y con comentarios explicativos.
2. El modelo en PgModeler representa fielmente las 6 relaciones uno-a-muchos del esquema original, con PK compuestas y FK que garantizan integridad referencial.

3. La migración de estructura con `mariadb-dump --no-data` + adaptación manual (`ENUM` → `CREATE TYPE`, limpieza de `ENGINE/CHARSET`, `int(11)` → `integer`, backticks → comillas dobles) y ejecución en `pdb_employees` vía `psql -f` fue verificada con `\dt`, `\d`, `\d+ e information_schema`.

6. Referencias

- Documentación MariaDB — `mariadb-dump`: <https://mariadb.com/kb/en/mariadb-dump/>
 - Documentación PostgreSQL 18 — `psql`, `CREATE TABLE`, `CREATE TYPE`: <https://www.postgresql.org/docs/>
 - PgModeler — Guía de uso y exportación: <https://pgmodeler.io/>
 - MySQL `employees` sample database — esquema de referencia: https://github.com/datacharmer/test_db
 - Presentación del Bloque 3 (Migración de SGBD) y guía de diferencias de tipos MariaDB vs PostgreSQL — Material_Actividad5_TBDI.
-

7. Anexo: archivos adjuntos

Archivo	Descripción
<code>Actividad_5_Informe.md</code>	Este informe en Markdown.
<code>Actividad_5_Informe.pdf</code>	Exportación a PDF del mismo informe.
<code>employees_diagrama.png</code>	Imagen PNG del diagrama ER de PgModeler (Punto 2).
<code>employees_postgres.sql</code>	Script DDL PostgreSQL exportado desde PgModeler (Puntos 2 y 3).

`unzip -l Actividad5_MarcoKiataque.zip`

```
Archive:  Actividad5_MarcoKiataque.zip
  Length   Date    Time    Name
  -----  -
  203576   2026-09-26  15:10   employees_diagrama.png
    1578   2026-09-26  11:41   employees_postgres.sql
    XXXX   2026-09-28  XX:XX   Actividad_5_Informe.md
    XXXX   2026-09-28  XX:XX   Actividad_5_Informe.pdf
```

Para empaquetar la entrega sin mover los originales:

```
cd ~/tecBD1
zip -j Actividad5_MarcoKiataque.zip employees_diagrama.png employees_postgres.sql
Actividad_5_Informe.md Actividad_5_Informe.pdf
```